

Modèle de données — Bibliothèque Universitaire (Université Omar Bongo)

Groupe 1 : MALEMBA Esther Lydie · OUSMANE-MOANDA Ali El-Hadj Mahamat · NDONG EYEGHE Georges Frédéric

Base : **MongoDB** · 3 collections : **ouvrages**, **adherents**, **emprunts**

1. Principe d'arbitrage embedding vs referencing

En modélisation documentaire, la question n'est pas « quelles sont les relations ? » (vision relationnelle) mais :

1. **Les données sont-elles lues ensemble ?** → si oui, candidates à l'embedding.
2. **La sous-collection est-elle bornée ?** → l'embedding exige une taille maîtrisée (limite document : 16 Mo, et performance dégradée sur les très gros tableaux).
3. **Les données ont-elles un cycle de vie propre ?** → si oui (consultées, filtrées, paginées indépendamment), candidates au referencing.
4. **A-t-on besoin de mises à jour atomiques ?** → MongoDB garantit l'atomicité **au niveau du document** : embarquer ce qui doit changer ensemble permet de gérer la concurrence sans transaction.

2. Décisions de modélisation

Donnée	Choix	Justification
Exemplaires d'un ouvrage	Embedded dans ouvrages	(a) Cardinalité bornée et faible : un ouvrage possède typiquement 1 à 30 exemplaires, jamais des milliers. (b) Lus ensemble : afficher un ouvrage = afficher la disponibilité de ses exemplaires (1 seule requête, pas de \$lookup). (c) Atomicité : changer le statut d'un exemplaire et décrémenter le compteur nbDisponibles se fait en une seule opération atomique sur le document — c'est la clé de la gestion de concurrence (voir §5).
Emprunts	Référencés (collection emprunts)	(a) Croissance non bornée : l'historique d'un ouvrage populaire peut atteindre des milliers d'emprunts → l'embarquer ferait grossir le document indéfiniment (anti-pattern <i>unbounded array</i>) jusqu'à la limite des 16 Mo. (b) Cycle de vie propre : on consulte les emprunts par adhérent, par période, par statut — indépendamment de l'ouvrage. (c) C'est une donnée d'événement (append-only) : idéale pour une collection dédiée, sur laquelle on agrège (top 10, statistiques).
Adhérents	Référencés (collection adherents)	Entité autonome (étudiants et enseignants) partagée par de nombreux emprunts : la dupliquer dans chaque emprunt créerait des anomalies de mise à jour. On dénormalise

Donnée	Choix	Justification
		seulement le nom dans l'emprunt (lecture rapide des listes sans \$lookup), car un nom change très rarement.
Réservation d'un exemplaire	Embedded dans l'exemplaire (reservePar , reserveLe)	La réservation est un état transitoire de l'exemplaire : la stocker dans l'exemplaire permet de la poser/lever atomiquement avec le changement de statut, ce qui empêche la double réservation.
Compteurs nbDisponibles , totalEmprunts	Dénormalisés dans ouvrages	Pattern <i>computed</i> : évite de recalculer la disponibilité à chaque affichage du catalogue. Maintenus par \$inc dans la même opération atomique que le \$set du statut.

3. Schémas des collections

ouvrages

```
{
  "_id": ObjectId,
  "titre": "Une vie de boy",
  "auteur": "Ferdinand Oyono",
  "isbn": "978-2-266-XXXXX",
  "annee": 1956,
  "categorie": "Littérature africaine",
  "nbDisponibles": 2,           // compteur dénormalisé ($inc)
  "totalEmprunts": 47,         // compteur dénormalisé ($inc)
  "exemplaires": [             // EMBEDDED – borné, lu avec l'ouvrage
    {
      "code": "UOB-0001-A",
      "etat": "bon",
      "statut": "disponible", // disponible | emprunte | reserve
      "reservePar": null,     // ObjectId adhérent si statut = reserve
      "reserveLe": null
    }
  ]
}
```

adherents

```
{
  "_id": ObjectId,
  "matricule": "UOB-ET-2024-153",
  "nom": "MALEMBBA", "prenom": "Esther",
  "type": "etudiant",         // etudiant | enseignant
  "email": "e.malemba@uob.ga"
}
```

emprunts (référence ouvrage + adhérent)

```
{
  "_id": ObjectId,
  "ouvrage": ObjectId,          // REFERENCE → ouvrages
  "exemplaireCode": "UOB-0001-A",
  "adherent": ObjectId,         // REFERENCE → adherents
  "adherentNom": "MALEMBA Esther", // dénormalisation de lecture
  "ouvrageTitre": "Une vie de boy",
  "dateEmprunt": ISODate,
  "dateRetourPrevue": ISODate,  // +14 j étudiant, +30 j enseignant
  "dateRetourEffective": null,
  "statut": "en_cours"          // en_cours | rendu
}
```

Index: emprunts {ouvrage:1},{adherent:1, statut:1}; ouvrages {"exemplaires.code":1}; adherents {matricule:1} unique.

4. Agrégation : top 10 des ouvrages les plus empruntés

```
db.emprunts.aggregate([
  { $group: { _id: "$ouvrage", totalEmprunts: { $sum: 1 } } },
  { $sort: { totalEmprunts: -1 } },
  { $limit: 10 },
  { $lookup: { from: "ouvrages", localField: "_id", foreignField: "_id", as:
"ouvrage" } } },
  { $unwind: "$ouvrage" },
  { $project: { _id: 0, titre: "$ouvrage.titre", auteur: "$ouvrage.auteur",
totalEmprunts: 1 } }
])
```

5. Gestion du statut et de la concurrence (\$set + \$inc)

Tout repose sur **findOneAndUpdate atomique** : le filtre vérifie la précondition (exemplaire disponible) et la mise à jour change le statut **dans la même opération**. Deux requêtes simultanées ne peuvent pas réussir toutes les deux : la seconde ne matche plus le filtre et reçoit **null** → l'API renvoie **409 Conflict**.

Emprunt (refusé si aucun exemplaire disponible)

```
const ouvrage = await Ouvrage.findOneAndUpdate(
  { _id: ouvrageId,
    exemplaires: { $elemMatch: { code, statut: "disponible" } } }, //
précondition
  { $set: { "exemplaires.$.statut": "emprunte" },
    $inc: { nbDisponibles: -1, totalEmprunts: 1 } },
  { new: true }
);
```

```
if (!ouvrage) return res.status(409).json({ erreur: "Aucun exemplaire disponible" });
```

Réservation (défi : empêcher la double réservation)

```
const ouvrage = await Ouvrage.findOneAndUpdate(
  { _id: ouvrageId,
    exemplaires: { $elemMatch: { code, statut: "disponible" } } }, // déjà
    réservé ⇒ ne matche pas
  { $set: { "exemplaires.$.statut": "reserve",
            "exemplaires.$.reservePar": adherentId,
            "exemplaires.$.reserveLe": new Date() },
    $inc: { nbDisponibles: -1 } },
  { new: true }
);
// 2 réservations simultanées : une seule passe, l'autre reçoit null → 409
```

Aucun verrou applicatif ni transaction multi-documents n'est nécessaire : c'est précisément **parce que les exemplaires sont embarqués** dans l'ouvrage que la précondition et la mutation tiennent dans un seul document, donc dans une seule opération atomique. Ce point relie le défi technique au choix de modélisation.